Tatweer Hackathon

How to Build a Winning Submission

Why this guide exists

This is your chance to grow. The strongest teams will not just be judged, they will be supported afterward through Athar+, the incubator we have partnered with, to take their idea further. So treat this submission as the start of something real, not just a competition entry. This guide shows you exactly what a winning submission looks like, in plain terms, so you can aim for it.

The one thing to understand

You are not judged on how flashy your project is. You are judged on whether you solve a real problem, prove it with evidence, and show it can survive hard questions. A simple project that is honest, tested, and well argued beats an ambitious one built on hype. Build for substance.

What a winning submission has

- **It is backed by evidence.** Do not just claim your idea helps people. Show why. Use real numbers, a small study, a survey of even ten people, a reference, or a test you ran. Every claim should have something behind it.
- **It has integrity.** Be honest about what works, what does not yet, and what you assumed. Judges trust teams that show their limits far more than teams that oversell. Do not fake results or hide gaps.
- **It is strong under rebuttal.** Imagine a judge pushing back: "Why would anyone use this?", "What if you are wrong?", "How do you know?" A winning submission has already answered these inside the README. Anticipate the hardest questions and address them head on.
- **It is creative.** Solve the problem in a way that is genuinely yours. Creativity is rewarded, but it must serve the problem, not replace substance.

Use the judging criteria as your checklist

We are sharing the full judging criteria with you on purpose. They are not a secret. Read them, then make sure your README earns points on every single one. If a criterion is not addressed anywhere in your repo, it is scored in its lowest band, so leave nothing out. The seven filtration criteria are: impact, relevance to the challenge, feasibility, readiness, scalability, evidence, and documentation.

A model README, section by section

Copy this structure into your repository README and fill each section honestly. The green note under each shows which criterion it earns.

1. The challenge and the problem

Earns: Relevance

State which of the five challenges you chose, then the specific problem inside it. Be precise. "People here cannot easily find local services" is sharper than "we help the community."

2. Who it is for, and their situation

Earns: Impact

Name the exact group affected and describe their situation in real terms. Who are they, how many, and what does the problem cost them today?

3. The solution

Earns: Relevance, Readiness

Explain what you built and how it solves the problem. Keep it clear enough that a non technical judge understands it in under a minute.

4. Impact and testable claims

Earns: Impact, Evidence

State the difference your solution makes as a claim that can be tested, then back it with evidence. Example: "In a test with 12 local users, 9 completed the task in under two minutes, versus none before." Specific and checkable beats grand and vague.

5. Feasibility and deployment

Earns: Feasibility

Show it can work in the real world. Address cost, who runs it, and what it takes to keep it running. Name the obstacles honestly rather than pretending there are none.

6. Scalability

Earns: Scalability

Explain how it grows beyond this weekend and reaches more people or other communities, and what that path looks like.

7. Evidence and validation

Earns: Evidence

Gather your proof in one place: data, tests, references, user feedback, screenshots. This is where integrity and strength under rebuttal live.

8. How to run or verify it, and tools used

Earns: Documentation, Readiness

Give clear steps to run or check your work, and list the tools you used. A judge should be able to verify it without contacting you. Include a short demo video.

Before you submit, ask yourself

- Does every claim I make have evidence behind it?
- Have I been honest about what does not work yet?
- If a judge attacks my weakest point, have I already answered it in the README?
- Could someone run or verify my project from the README alone?
- Have I addressed all seven criteria somewhere in the repo?